CHAPITRE X

POO: LES ENTREES/SORTIES: (FLUX)

Présentation des FLux

- Les flux (stream en anglais) permettent d'encapsuler des processus d'envoi et de réception de données. Les flux traitent toujours les données de façon séquentielle.
- En java, les flux peuvent être divisés en plusieurs catégories :
  - les flux d'entrée (input stream) et les flux de sortie (output stream).
  - les flux de traitement de caractères et les flux de traitement d'octets.
- Java définit des flux pour lire ou écrire des données mais aussi des classes qui permettent de faire des traitements sur les données du flux. Ces classes doivent être associées à un flux de lecture ou d'écriture et sont considérées comme des filtres. Par exemple, il existe des filtres qui permettent de mettre les données traitées dans un tampon (buffer) pour les traiter par lots.

Toutes ces classes sont regroupées dans le package java.io.

Les Classes de gestion des FLux

- L'utilisation de ces classes n'est pas chose facile, vu leur nombre élevé et la difficulté de choisir la classe convenable à un besoin précis.
- Pour bien choisir une classe adaptée à un traitement donné, il faut comprendre la dénomination des différentes classes.
- Le nom d'une classe est toujours composé d'un **préfixe** et d'un **suffixe**.. Il y a quatre suffixes possibles en fonction du type de flux (flux d'octets ou de caractères) et du sens du flux (entrée ou sortie).

	Flux d'octets	Flux de caractères
Flux d'entrées	InputStream	Reader
Flux de sorties	OutputStream	Writer

Les Classes de gestion des FLux

Il existe donc quatre hiérarchies de classes qui encapsulent des flux particuliers.
Ces classes peuvent être séparées en deux séries de deux catégories différentes: les classes de lecture et d'écriture et les classes de manipulation de caractères et d'octets.

- ✓ les sous classes de **Reader** sont des types de flux en **lecture** sur les **caractères,**
- ✓ les sous classes de **Writer** sont des types de flux en **écriture** sur les **caractères,**
- ✓ les sous classes de **InputStream** sont des types de flux en **lecture** sur les **octets**,
- ✓ les sous classe de **OutputStream** sont des types de flux en **écriture** sur les **octets**.

Les Classes de gestion des FLux

On peut distinguer un ensemble de préfixes qui contiennent le type de traitement qu'il effectue.

Buffered : ce type de filtre permet de mettre les données du flux dans un tampon. Il peut être utilisé en entrée et en sortie.
Sequence : ce filtre permet de fusionner plusieurs flux.
Data : ce type de flux permet de traiter les octets sous forme de type de données.
LineNumber : ce filtre permet de numéroter les lignes contenues dans le flux.
PushBack : ce filtre permet de remettre des données lues dans le flux.
Print : ce filtre permet de réaliser des impressions formatées.
Object : ce filtre est utilisé par la sérialisation
InputStream / OuputStream : ce filtre permet de convertir des octets en caractères.

Buffered
Sequence
Data
LineNumber
PushBack
Print
Object
OutputStream

Les Flux de caractères en lecture/écriture

Ils transportent des données sous forme de caractères : java les gèrent avec le format Unicode qui code les caractères sur 2 octets. Les classes qui gèrent les flux de caractères héritent d'une des deux classes abstraites Reader ou Writer. Il existe de nombreuses sous classes pour traiter les flux de caractères.

	Flux en lecture	Flux en sortie
Flux de caractères	BufferedReader CharArrayReader FileReader InputStreamReader LineNumberReader PipedReader PushbackReader StringReader	BufferedWriter CharArrayWriter FileWriter OutputStreamWriter PipedWriter StringWriter

La classe Reader

C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de caractères en lecture.
Cette classe définit plusieurs méthodes :

Méthodes	Rôles
boolean markSupported()	indique si le flux supporte la possibilité de marquer des positions
boolean ready()	indique si le flux est prêt à être lu
close()	ferme le flux et libère les ressources qui lui étaient associées
int read()	renvoie le caractère lu ou -1 si la fin du flux est atteinte.
int read(char[])	lire plusieurs caractères et les mettre dans un tableau de caractères
int read(char[], int, int)	lire plusieurs caractères. Elle attend en paramètre : un tableau de caractères qui contiendra les caractères lus, l'indice du premier élément du tableau qui recevra le premier caractère et le nombre de caractères à lire. Elle renvoie le nombre de caractères lus ou -1 si aucun caractère n'a été lu. Le tableau de caractères contient les caractères lus.

La classe FileReader

Cette classe permet de gérer les fichiers texte d'entrée. Elle dispose des mêmes fonctionnalités que la classe abstraite Reader.

Constructeur	Rôle
FileInputReader(String)	Créer un flux en lecture vers le fichier dont le nom est précisé en paramètre.
FileInputReader(File)	Idem mais le fichier est précisé avec un objet de type File

```
FileReader fichier = new FileReader("fichier.txt");
```

Il existe plusieurs méthodes de la classe FileReader qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes sont héritées de la classe Reader et peuvent toutes lever l'exception IOException.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode **close().**

La classe Writer

C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de caractères en écriture.
Cette classe définit plusieurs méthodes :

Méthodes	Rôles
close()	ferme le flux et libère les ressources qui lui étaient associées
write(int)	écrire le caractère en paramètre dans le flux.
write(char[])	écrire le tableau de caractères en paramètre dans le flux.
write(char[], int, int)	écrire plusieurs caractères. Elle attend en paramètres : un tableau de caractères, l'indice du premier caractère dans le tableau à écrire et le nombre de caractères à écrire.
write(String)	écrire la chaîne de caractères en paramètre dans le flux
write(String, int, int)	écrire une portion d'une chaîne de caractères. Elle attend en paramètre : une chaîne de caractères, l'indice du premier caractère dans la chaîne à écrire et le nombre de caractères à écrire.

La classe FileWriter

Cette classe permet de gérer les fichiers texte d'entrée. Elle dispose des mêmes fonctionnalités que la classe abstraite Writer.

Constructeur	Rôle
FileWriter(String)	Si le nom du fichier précisé n'existe pas alors le fichier sera créé. Si il existe et qu'il contient des données celles-ci seront écrasées.
FileWriter(File)	Idem mais le fichier est précisé avec un objet de la classe File.
FileWriter(String, boolean)	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur true) ou écraseront les données existantes (valeur false)

```
FileWriter fichier = new FileWriter("fichier.txt");
```

Il existe plusieurs méthodes de la classe FileWriter qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes sont héritées de la classe Writer et peuvent toutes lever l'exception IOException.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode **close().**

Les Flux de caractères tamponnés:

Pour améliorer les performances des flux sur un fichier, la mise en tampon des données lues ou écrites permet de traiter un ensemble de caractères représentant une ligne plutôt que de traiter les données caractères par caractères. Le nombre d'opérations est ainsi réduit.
Les classes **BufferedReader** et **BufferedWriter** permettent de gérer des flux de caractères tamponnés avec des fichiers.

La classe BufferedReader

Cette classe possède plusieurs constructeurs qui peuvent tous lever une exception de type FileNotFoundException:

Constructeur	Rôle
BufferedReader(Reader)	le paramètre fourni doit correspondre au flux à lire.
BufferedReader(Reader, int)	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception de type IllegalArgumentException est levée.

```
BufferedReader fichier = new BufferedReader(new FileReader("fichier.txt"));
```

La classe BufferedReader

Il existe plusieurs méthodes de la classe BufferedReader héritées de la classe Reader qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes peuvent lever une exception de type IOException. Elle définit une méthode supplémentaire pour la lecture :

Méthode	Rôle
String readLine()	lire une ligne de caractères dans le flux. Une ligne est une suite de caractères qui se termine par un retour chariot '\r' ou un saut de ligne '\n' ou les deux.

La classe BufferedReader possède plusieurs méthodes pour gérer le flux hérité de la classe Reader.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Exemple de lecture d'un fichier texte

```
import java.io.*; public class TestBufferedReader {

public void lecture() {
 try {
String ligne ;
   FileReader fr=new FileReader(source);
BufferedReader  br = new BufferedReader(fr);
while ((ligne = br.readLine()) != null)
{ System.out.println(ligne); }
 br.close();
fr.close(); }
 catch (Exception e)
 { e.printStackTrace(); }} }

Public class TestBR
public static void main(String args[]) {
 TestBufferedReader tb=new TestBufferedReader();
tb.lecture(); }
```

La classe BufferedWriter

Cette classe possède plusieurs constructeurs Exemple:

Constructeur	Rôle
BufferedWriter(Writer)	le paramètre fourni doit correspondre au flux dans lequel les données sont écrites.
BufferedWriter(Writer, int)	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception IllegalArgumentException est levée.

```
BufferedWriter fichier = new BufferedWriter( new FileWriter("monfichier.txt"));
```

Exemple d'écriture dans un fichier texte

```
import java.io.*; import java.util.*;
 public class TestBufferedWriter
{
private void traitement() {
try { String ligne ; int nombre = 123;
FileWriter fw=new FileWriter(destination);
BufferedWriter bw = new BufferedWriter(fw);
bw.write("bonjour tout le monde");
bw.newLine();
bw.write("Nous sommes le "+ new Date());
bw.write(", le nombre magique est " + nombre);
 fw.close(); } catch (Exception e)
 { e.printStackTrace(); } } }


Public class TestBW
public static void main(String args[])
 {TestBufferedWriter tbw=new TestBufferedWriter();
            tbw.traitement(); }
```

La classe PrintWriter

Cette classe permet d'écrire dans un flux des données formatées.
Cette classe possède plusieurs constructeurs :

Constructeur	Rôle
PrintWriter(Writer)	Le paramètre fourni précise le flux. Le tampon est automatiquement vidé.
PrintWriter(Writer, boolean)	Le booléen permet de préciser si le tampon doit être automatiquement vidé
PrintWriter(OutputStream)	Le paramètre fourni précise le flux. Le tampon est automatiquement vidé.
PrintWriter(OutputStream, boolean)	Le booléen permet de préciser si le tampon doit être automatiquement vidé

La classe PrintWriter

```
PrintWriter fichier = new PrintWriter( new FileWriter("monfichier.txt"));
```

Il existe de nombreuses méthodes de la classe PrintWriter qui permettent d'écrire un ou plusieurs caractères dans le flux en les formatant. Les méthodes write() sont héritées de la classe Writer. Elle définit plusieurs méthodes pour envoyer des données formatées dans le flux :

print(...)

Plusieurs méthodes print acceptent des données de différents types pour les convertir en caractères et les écrire dans le flux

println()

Cette méthode permet de terminer la ligne courante dans le flux en y écrivant un saut de ligne.

println (...)

Plusieurs méthodes println acceptent des données de différents types pour les convertir en caractères et les écrire dans le flux avec une fin de ligne.

La classe PrintWriter possède des méthodes pour gérer le flux :

Méthode	Rôle
flush()	Vide le tampon en écrivant les données dans le flux.

Exemples

```
import java.io.*;
 import java.util.*;
public class TestPrintWriter {
private void traitement() {
try { String ligne ; int nombre = 123;
PrintWriter fichier = new PrintWriter(new FileWriter(destination));
fichier.println("bonjour tout le monde");
fichier.println("Nous sommes le "+ new Date());
fichier.println("le nombre magique est " + nombre);
fichier.close(); } catch (Exception e)
{ e.printStackTrace(); } }

public static void main(String args[]) {
TestPrintWriter  tp=new TestPrintWriter();  tp.traitement();}
}
```

Les Flux d’octets

Ils transportent des données sous forme d'octets. Les flux de ce type sont capables de traiter toutes les données.
Les classes qui gèrent les flux d'octets héritent d'une des deux classes abstraites InputStream ou OutputStream. Il existe de nombreuses sous classes pour traiter les flux d'octets.

Les flux d'octets avec un fichier.

Les classes FileInputStream et FileOutputStream permettent de gérer des flux d'octets avec des fichiers.

Les flux d'octets en lecture sur un fichier

Il faut instancier un objet de la classe FileInputStream. Cette classe possède plusieurs constructeurs qui peuvent tous lever l'exception FileNotFoundException:

Constructeur	Rôle
FileInputStream(String)	Ouvre un flux en lecture sur le fichier dont le nom est donné en paramètre
FileInputStream(File)	Idem mais le fichier est précisé avec un objet de type File

```
FileInputStream fichier = new FileInputStream("monfichier.dat");
```

Les flux d'octets en lecture sur un fichier

Il existe des méthodes de la classe FileInputStream qui permettent de lire un ou plusieurs octets dans le flux. Toutes ces méthodes peuvent lever l'exception IOException.

int read( )

Cette méthode envoie la valeur de l'octet lu ou -1 si la fin du flux est atteinte.

int read(byte[], int, int), int read(byte[ ])

Cette méthode lit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contiendra les octets lus, l'indice du premier élément du tableau qui recevra le premier octet et le nombre d'octets à lire.

Elle renvoie le nombre d'octets lus ou -1 si aucun octet n'a été lu. Le tableau d'octets contient les octets lus.

Les flux d'octets en lecture sur un fichier

La classe FileInputStream possède plusieurs méthodes pour gérer le flux :

Méthode	Rôle
long skip(long)	saute autant d'octets dans le flux que la valeur fournie en paramètre. Elle renvoie le nombre d'octets sautés.
close()	ferme le flux et libère les ressources qui lui étaient associées
int available()	retourne une estimation du nombre d'octets qu'il est encore possible de lire dans le flux

Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Les flux d'octets en écriture sur un fichier

Il faut instancier un objet de la classe FileOutputStream. Cette classe possède plusieurs constructeurs :

Constructeur	Rôle
FileOutputStream(String)	Si le fichier précisé n'existe pas, il sera créé. Si il existe et qu'il contient des données celles-ci seront écrasées.
FileOutputStream(String, boolean)	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur true) ou écraseront les données existantes (valeur false)

```
FileOuputStream fichier = new FileOutputStream("monfichier.dat");
```

Les flux d'octets en écriture sur un fichier

Il existe plusieurs méthodes de la classe FileOutputStream qui permettent de lire un ou plusieurs octets dans le flux.

write(int)

Cette méthode écrit l'octet en paramètre dans le flux.

write(byte[])

Cette méthode écrit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contient les octets à écrire : tous les éléments du tableau sont écrits.

write(byte[], int, int)

Cette méthode écrit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contient les octets à écrire, l'indice du premier élément du tableau d'octets à écrire et le nombre d'octets à écrire.

Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Exemple

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class CopieFichier {
public static void main(String args[]) {
try { copierFichier("source.txt", "copie.txt"); }
catch (IOException e) { e.printStackTrace(); } }
 public static void copierFichier(String source, String destination) throws
IOException {
FileInputStream fis = null;
FileOutputStream fos = null; try { byte buffer[] = new byte[1024];
int taille = 0;
fis = new FileInputStream(source);
fos = new FileOutputStream(destination);
while ((taille = fis.read(buffer)) != -1)
{ System.out.println(taille);
fos.write(buffer, 0, taille); } }
 finally { if (fis != null) { try { fis.close(); } catch (IOException e) { } } if
(fos != null) { try { fos.close(); } catch (IOException e) { } } } } }
```

La classe File

Une instance de la classe File est une représentation logique d'un fichier ou d'un répertoire qui peut ne pas exister physiquement sur le disque.
Si le fichier ou le répertoire existe, de nombreuses méthodes de la classe File permettent d'obtenir des informations sur le fichier. Sinon plusieurs méthodes permettent de créer des fichiers ou des répertoires. Voici une liste de quelques méthodes :

Méthode	Rôle
boolean canRead()	indique si le fichier peut être lu
boolean canWrite()	indique si le fichier peut être modifié
boolean exists()	indique si le fichier existe physiquement
String getAbsolutePath()	renvoie le chemin absolu du fichier
String getPath	renvoie le chemin du fichier
boolean isDirectory()	indique si le fichier est un répertoire
long length()	renvoie la longueur du fichier
String[] list()	renvoie la liste des fichiers et répertoire contenu dans le rép
boolean mkdir()	création du répertoire
boolean mkdirs()	création du répertoire avec création des répertoires manquants dans l'arborescence du chemin

Utilisation de quelques méthodes

```
import java.io.*;
 public class TestFile {
 protected String nomFichier;
protected File fichier;
 public TestFile(String nomFichier) { this.nomFichier = nomFichier;
 fichier = new File(nomFichier);
 traitement(); }
private void traitement() {
 if (!fichier.exists()) {
 System.out.println("le fichier "+nomFichier+"n'existe pas");
 System.exit(1); }
 System.out.println(" Nom du fichier : "+fichier.getName());
 System.out.println(" Chemin du fichier : "+fichier.getPath());
 System.out.println(" Chemin absolu : "+fichier.getAbsolutePath());
 System.out.println(" Droit de lecture : "+fichier.canRead());
 System.out.println(" Droit d'ecriture : "+fichier.canWrite());
 if (fichier.isDirectory())
 { System.out.println(" contenu du repertoire ");
String fichiers[] = fichier.list();
for(int i = 0; i < fichiers.length; i++)
System.out.println(" "+fichiers[i]); } } }
```

```
 public class TestMethodesFile
public static void main(String args[]) {
 new TestFile(args[0]); }
```

Les fichiers à accès direct

Les fichiers à accès direct permettent un accès rapide à un enregistrement contenu dans un fichier.

La classe RamdonAccessFile encapsule les opérations de lecture/écriture d'un tel fichier.
Elle implémente les interfaces **DataInput** et **DataOutput.**
Elle possède deux constructeurs qui attendent en paramètre **le fichier à utiliser** (sous la forme d'un nom de fichier ou d'un objet de type File qui encapsule le fichier) et **le mode d'accès**.
Le mode est une chaîne de caractères qui doit être égal à **«r»** ou **«rw»** selon que le mode est lecture seule ou lecture/écriture.
Ces deux constructeurs peuvent lever les exceptions suivantes :
FileNotFoundException si le fichier n'est pas trouvé
IllegalArgumentException si le mode n'est pas «r» ou «rw»
SecurityException si le gestionnaire de sécurité empêche l'accès aux fichiers dans le mode précisé

Les fichiers à accès direct

La classe RandomAccessFile possède de nombreuses méthodes writeXXX() pour écrire des types primitifs dans le fichier

```
package 1
import java.io.RandomAccessFile;
public class TestRandomAccesFile {
public static void main(String[] args) {
try {
RandomAccessFile monFichier = new RandomAccessFile("monfichier.txt", "rw");
for (int i = 0; i < 10; i++)
{ monFichier.writeInt(i * 100); }
monFichier.close(); }
catch (Exception e)
{ e.printStackTrace(); } } }
```

Les fichiers à accès direct

Elle possède aussi de nombreuses classes readXXX() pour lire des données primitives dans le fichier.

```
package 1
import java.io.RandomAccessFile;
public class TestRandomAccesFile {
public static void main(String[] args) {
try {
RandomAccessFile monFichier = new RandomAccessFile("monfichier.txt", "rw"); for (int i =
0; i < 10; i++)
{ System.out.println(monFichier.readInt()); }
monFichier.close(); }
catch (Exception e)
{ e.printStackTrace(); } } }
```

CHAPITRE XI

GUI: LES INTERFACES GRAPHIQUES (JavaFx)

HISTORIQUE(Chronologie)

1995	1996	1998	2007	2011	2014-2018
AWT	**JAVA1.0 SWING**	**JAVA1.2**	**Annonce de JavaFX**	**Java7**	**Java8-JavaFX**

A l'origine du langage Java, les interfaces graphiques étaient créées en utilisant la librairie AWT (java.awt)

Rapidement, la librairie Swing (javax.swing) est venu compléter (et partiellement remplacer) la librairie AWT

HISTORIQUE(SWING vs AWT)

- Tous les composants du package AWT sont des composants lourds.
- Les composants Swing (nom du package : javax.swing) sont pour la majorité d'entre eux des composants légers.
- Tous les composants Swing commencent par la lettre " J ".
- Les composants Swing se répartissent :
  - en conteneurs de plus lourd:
    - **JFrame** à rapprocher de la classe **Frame** dans AWT
    - **JDialog** à rapprocher de la classe **Dialog** dans AWT
    - **JWindow** à rapprocher de la classe **Window** dans AWT
    - **JApplet** à rapprocher de la classe **Applet** dans AWT
  - en conteneurs légers (les autres contrôles GUI Swing).

HISTORIQUE(SWING vs AWT)

JavaFX 1	JavaFX 2	JavaFX 8
a tenté, sans grand succès, de remplacer Swing	Pas de script. Java API et/ou XML:FXML SceneBuilder, CSS	Plus de librairie externe à télécharger et à référencer **Scene Builder 2 , lambda** et les streams ,DatePicker TreeTableView. Gestion des écrans tactiles (TouchEvent,GestureEvent,) Amélioration des librairies graphiques 2D et 3D ,CSS

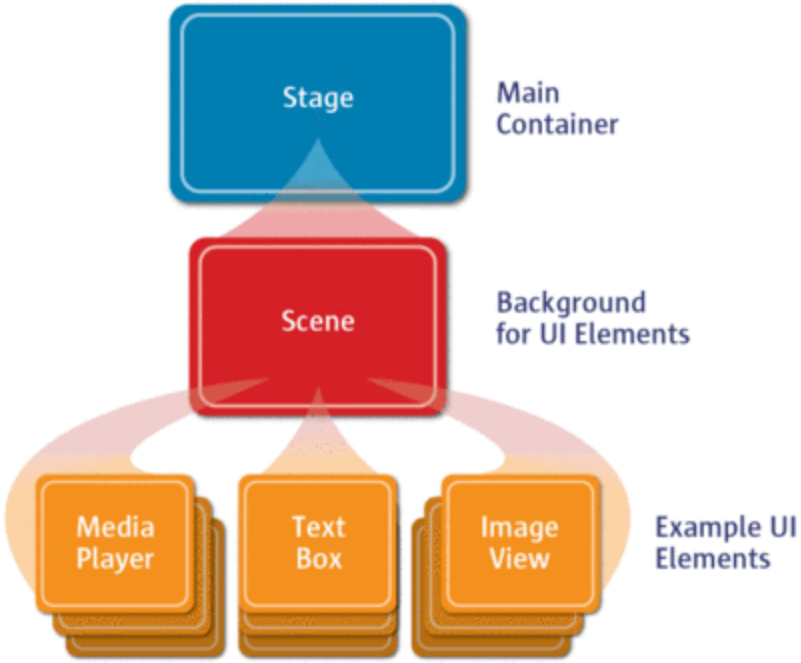
Presentation d'une application JavaFX

Pour construire une application JavaFX il faut:

- ➢Hériter de la classe Application.

- ➢Créer un objet de type **Stage** pour le lancement de l'application.

- ➢Créer un objet de type **Scene** et l'associer à la fenêtre(Stage).

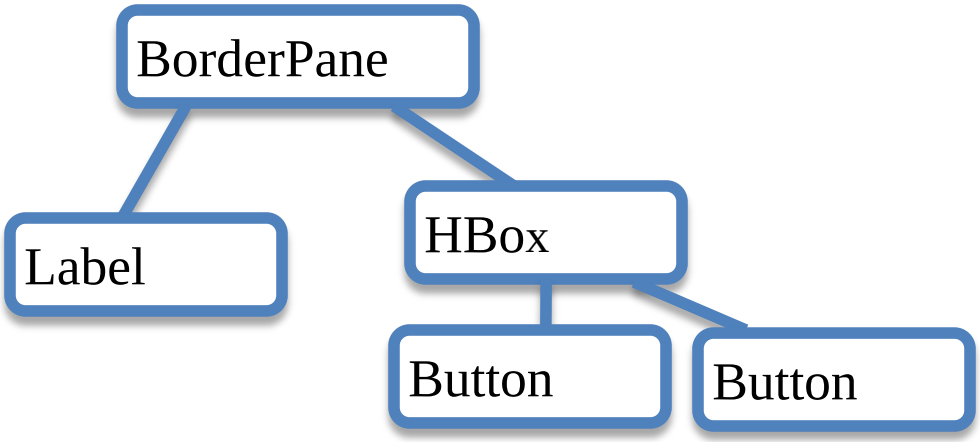


- ➢La Scène est composée de nœuds (**Node**)qui représente les différents éléments de l'interface graphique.

- ➢Enfin on appel la méthode start() qui construit l'interface.

Graphe de Scene

Forme primitive (**Shape**)	Conteneurs(**Layout-Pane**)	Composant standard (**Control**)	Composants spécialisés
Line, Circle,Rectangle, Box, Cylinder	AnchorPane, StackPane, BorderPane, GridPane, HBox, Vbox,	Label, Button, TetField, ComboBox	MediaView, WebView, ImageView,



Creation d'une fenetre personnalisee

```
public class PremiereFenetre extends Application {
Button bouton;
public void start(Stage premierStage)
{
    premierStage.setTitle("Ma premiere application javaFx");
    bouton=new Button();
    bouton.setText("Click me");
    StackPane root=new StackPane();
    root.getChildren().add(bouton);
    Scene scene=new Scene(root,300,250);
    premierStage.setScene(scene);
    premierStage.show();
}
public static void main(String[] args) {
launch(args);}}
```

Action sur un Bouton

```
public class PremiereFenetre extends Application
implements EventHandler<ActionEvent>{
Button bouton;
public void start(Stage premierStage)
{
        premierStage.setTitle("Ma premiere application javaFx");
        bouton=new Button();
        bouton.setText("Click me");
        StackPane root=new StackPane();
        root.getChildren().add(bouton);
        Scene scene=new Scene(root,300,250);
        bouton.setOnAction(this);
        premierStage.setScene(scene);
        premierStage.show();}
        public static void main(String[] args) {
        launch(args);}
public void handle(ActionEvent event) {
if(event.getSource()==bouton)
System.out.println("Vous m'avez touche");}}
```

La méthode **gerSource** fournit une référence sur l'objet ayant déclenché l'événement concerné.

Classe Anonyme

Il est possible de définir une classe interne, sans lui donner de nom par dérivation d'une super classe, ou par implémentation d'une interface.

```
bouton.setOnAction(new
EventHandler<ActionEvent>(){
public void handle(ActionEvent event) {
if(event.getSource()==bouton)
System.out.println("Vous m'avez touche"); }});
```

Les expressions Lamda

```
bouton.setOnAction(e->System.out.println("Vous m'avez touche"));
```

Les expressions **lambd**a sont aussi nommées **closures** ou fonctions anonymes : leur but principal est de permettre de passer en paramètre un ensemble de traitements.

Les événements clavier

- **onKeyPressed:** touche enfoncée
- **onKeyReleased:** touche relâchée
- **onKeyTyped:** touche saisie

Pour recevoir des événements il faut obtenir le focus. On doit mettre à true le focusTransversable

Les événements clavier

```
public class EvenementClavierSouris extends Application{
//Button bouton;
public void start(Stage premierStage)
{
Circle cercle=new Circle();
cercle.setCenterX(100);
cercle.setCenterY(100);
cercle.setRadius(50);
cercle.setFill(Color.AQUAMARINE);
cercle.setFocusTraversable(true);
cercle.setOnKeyPressed(KeyEvent->System.out.prinln("Touche enfoncee: :"+KeyEvent.getCode()+" "
+KeyEvent.getCharacter()));
cercle.setOnKeyReleased(KeyEvent->System.out.println("Touche relachee:"+KeyEvent.getCode()+" "
+KeyEvent.getCharacter()));
cercle.setOnKeyTyped(KeyEvent->System.out.println("touche saisie: "+KeyEvent.getCode()+ " "
+KeyEvent.getCharacter()));
premierStage.setTitle("Evenement clavier");
Pane root=new Pane();
root.getChildren().setAll(cercle);
Scene scene=new Scene(root,300,250,Color.GREEN);
premierStage.setScene(scene);
premierStage.show();}
public static void main(String[] args) {
launch(args);}}
```

Les événements souris

- **onMouseClicked:** clic de souris
- **onMouseDragged:** la souris cliquée-tiree
- **onMouseEntered:** la souris est entrée dans la zone du nœud
- **onMouseExited:** la souris est sortie de la zone
- **onMouseMoved:** la souris est deplacee
- **onMousePressed:** la souris est enfoncée
- **onMousereleased:** la souris est relâchée

On n'a pas besoin de mettre à true le focusTransversable pour obtenir le focus.

Les événements souris

cercle.setOnMouseClicked(MouseEvent->System.***out.println("Vous avez clique sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseDragged(MouseEvent->System.***out.println("Vous avez effectue un drag'n drop sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseEntered(MouseEvent->System.***out.println("Vous avez entre la souris sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseExited(MouseEvent->System.***out.println("Vous avez sorti la souris au point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseMoved(MouseEvent->System.***out.println("Vous avez deplace sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMousePressed(MouseEvent->System.***out.println("Vous avez appuye la souris sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseClicked(MouseEvent->System.***out.println("Vous avez relache la souris au point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***